PayAudit: A Central Bank Digital Currency with Offline Payment and Post-Hoc Auditability

Cash-Like Privacy. Auditable only after settlement.

Wong Sze Ho - Supervisor: Prof. Sherman Chow

Department of Information Engineering, The Chinese University of Hong Kong

Introduction and Motivation

Goal: **Offline Private Central Bank Digital Currency (CBDC)**

- **Offline Payments** – users should be able to transact without network access.
- **Cash-like Privacy** – no one should automatically see your balance or what you bought.
- **Accountability when needed** – authorities should be able to investigate suspicious transactions under strict authorization.

Scheme	Offline	Audit & Trace	Mechanism / Limitation
PEReDi / Platypus	✗	✓	Post-hoc audit via threshold ElGamal; **must be online** during payment.
PayOff	✓	✗	Offline chained settlement; resists TEE compromise but **no audit capability**.
PayAudit (Ours)	✓	✓	**First to combine both:** adds audit ciphertext (ψ) and traceable double-spend tag (T) to PayOff.

Figure 1: Comparision Table

PayOff [1] achieves offline privacy but offers no post-hoc audit. TEE-backed designs provide accountability but rely on a trusted hardware manufacturer — a supply-chain assumption unacceptable for a sovereign CBDC.

PayAudit achieves all three in pure cryptography
(no trusted hardware needed)

Offline Payment Protocol

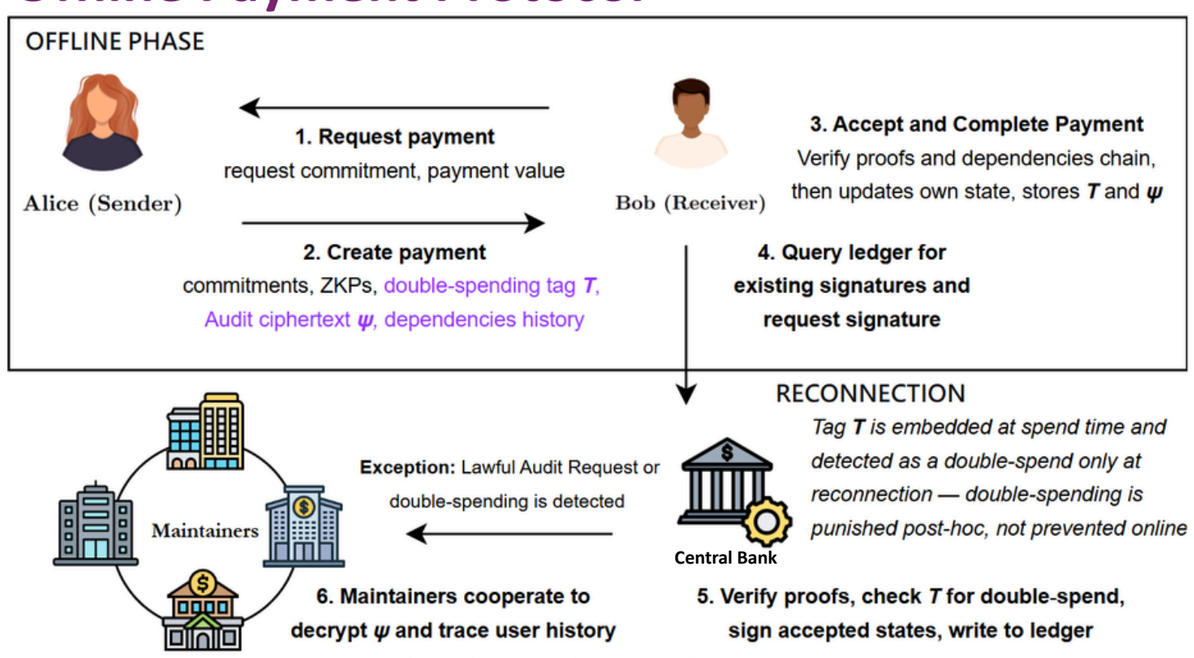


Figure 2: PayAudit protocol flow

Performance and Contributions

Fully implemented in Go — end-to-end proof generation and verification

Metric	Value
Single offline payment (create + accept + complete)	< 0.3 s
Settlement of a 100-payment chain	< 0.6 s
Message size for 100-payment history	~ 105 KB
Circuit-size comparison (creation proof)	
ElGamal & tag computed inside the ZK circuit	33 714 constraints
ElGamal & tag computed outside the circuit (+ Schnorr proof)	19 306 constraints
→ constraint reduction	$\sim 43\%$
Schnorr proof generation (added cost)	0.0006 s (negligible)
Schnorr proof verification (added cost)	0.0012 s (negligible)

Tested on a laptop with AMD Ryzen 7 4800H CPU @ 2.90GHz (Zen 2, 16 GB)

Figure 4: PayAudit Performance results

- **PayOff** [1] similar creation ≈ 0.21 s on a laptop with *AMD Ryzen 7 PRO 7730U processor (Zen 3, 32 GB) (arXiv:2408.06956, Table III)*; ours **≈ 0.12 s** with added audit/trace. Direct comparison is approximate; **our older CPU makes the speedup conservative**.
- Performance identical to a scheme without audit (negligible Schnorr cost).
- 100 offline payments produce ~105 KB of proof data, ~1 KB per payment — comparable to a compressed image, well within mobile storage.

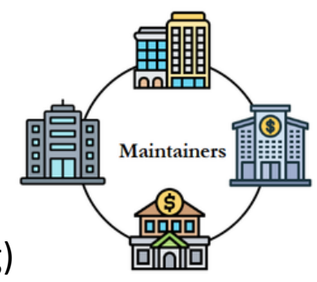
Three key contributions

1. **First unified account-based design (balance per user, not per coin)** – combines offline chained settlement with threshold-based post-hoc audit in a single scheme.
2. **Efficient cryptography** – we separate the expensive wallet-linking step from proof generation. A compact Schnorr proof (≈64 bytes) bridges them, achieving mobile-speed proofs without sacrificing security.
3. **Balanced privacy vs. regulation** – everyday payments are cash-like anonymous; lawful access requires distributed authorization, preventing unilateral surveillance.

Summary

PayAudit separates ordinary payment verification from exceptional audit — Central Bank sees no private data, while authorised maintainers can only decrypt transactions under lawful order.

System Overview



Actors:

User (payer/payee)

Central Bank(CB) (verifies proofs, check double-spending)

Maintainers / Authorities (any any 3 of 5 must cooperate to audit)

1. Committed Wallet State — Balance hidden inside a cryptographic commitment

↓

2. Zero-Knowledge Update — Prove correct deduction & no overspend, without revealing the balance

↓

3. Embedded Tag *T* & Audit ciphertext ψ in payment — **Audit Ciphertext ψ**: threshold-locked amount & identities; **Traceable Tag *T***: detects double-spend, anchors user tracing

↓

4. Delayed Settlement — CB verifies proofs & checks ***T*** for double-spend.

Security & Privacy Mechanisms

Requirement	Mechanism	What It Means
Hide balance	Committed wallet state	Balance is hidden in a cryptographic commitment on the ledger.
Prevent overspending	Zero-knowledge proof	Proves a non-negative resulting balance without revealing values.
Detect double-spending	Deterministic tag $T = g^{a \cdot \mathrm{ctr}}$	Reusing an old state produces the same T, allowing the bank to detect duplicates.
Hide identities	Threshold ElGamal ψ	Transaction details are locked; no single party can open them.
Enable lawful audit	$(t+1)$ Threshold Decryption	Only a quorum of maintainers can unlock ψ to reveal details.
Trace user history	Secret key a reconstruction	Regulator uses secret a to compute all possible T and scan the history.
Prevent tampering	Dependency chain	Each state links to the previous; any chain break is immediately detectable.

Figure 3: PayAudit Security overview

Threat Model

- **5 maintainers**; any **3** must cooperate to audit or trace.
- At most **2** may be malicious or unavailable. (i.e., t = 2, 5t+1 = 5)
- CB is trusted to verify transactions correctly, but **NOT** trusted to protect user privacy.
- Even if CB colludes with **all malicious maintainers (up to 2)**, privacy remains intact.

References

[1] C. Beer et al., PayOff: A Regulated Central Bank Digital Currency with Private Offline Payments, AsiaCCS, 2026.

[2] A. Sarencheh et al., PEReDi: Privacy-Enhanced, Regulated and Distributed Central Bank Digital Currencies, ACM CCS, 2022.

[3] K. Wüst et al., Platypus: A Central Bank Digital Currency with Unlinkable Transactions and Privacy-Preserving Regulation, ACM CCS, 2022.

Go prototype (gnark/Groth16, BN254)